

Groundwork

Turning scattered client input — recorded calls, chat exports, contracts, screenshots — into a plan you can defend, where every line is traceable back to the sentence somebody actually said.

Ashika · 18 August 2026 · AI Business Discovery to POC

This is a working tool, not a slide deck, but it is deliberately not a finished product. The brief said the goal was to show how an open business problem is turned into a working solution in limited time, so I built the parts that carry the idea and left the parts that only carry weight in production. Where I stopped, I have said so plainly — including a section on what can go wrong when you run it, and why.

SECTION ONE

Resources

SOURCE CODE

<https://github.com/malicious28/groundwork>

Public. The commit history is worth a look — every feature was built on its own branch and merged separately, so `git log --graph --first-parent` reads as a sequence of decisions rather than one drop of code.

ZIP

[groundwork.zip](#)

The same code, attached to the email. Identical to the repository except that a zip cannot carry git history.

VIDEO — PART 1

<https://www.loom.com/share/1c8385c26338433e8ff7b273c45a99c5>

VIDEO — PART 2

<https://www.loom.com/share/afb2c7b02aae4825ad0c332652abe097>

PLAIN-LANGUAGE EXPLAINER

[docs/groundwork-explained.pdf](#)

19 pages, written for a non-technical reader. Covers the whole product and why each decision was made.

BUILD PLAN

[docs/groundwork-plan.html](#)

The plan and the market survey the build started from, written before any code.

Running it

No Docker, no database server, no cloud account, and no API key needed to see the whole thing working. The database is Postgres compiled to WebAssembly and runs inside the application process, so setup is genuinely two commands.

```
# Node 20 or newer
npm install
cp .env.example .env
```

```
# AUTH_SECRET is required – it signs the session cookie. Generate one:
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
# paste the output after AUTH_SECRET= in .env

npm run setup      # creates the database, applies security policies, loads the demo
npm run dev        # http://localhost:3000
```

Sign in with `ashika@meridian.example` and password `demo1234`. The sign-in screen carries the credentials, so there is nothing to look up. Two other accounts are seeded: a client who sees only what was shared with them, and an unrelated firm that proves it can see none of it.

The AI key is optional, and this matters

Without `ANTHROPIC_API_KEY`, the app replays a recorded analysis of the seeded project — but **the verification layer still runs for real** against the actual documents. That is why the demo can show a claim failing its check. With a key set, the same code path calls the model.

`npm run set-key` writes the key into `.env` correctly; the server must be restarted afterwards, because `.env` is read once at startup.

COMMAND	WHAT IT DOES
<code>npm run dev</code>	Start the app
<code>npm run db:reset</code>	Wipe local data and reload the demo — safe, and the fastest fix for most problems
<code>npm run db:fresh</code>	An empty workspace, to walk the flow with your own documents
<code>npm test</code>	230 automated tests
<code>npm run test:e2e</code>	12 tests driving a real browser through the demo

What it does, and how the AI works

A client never explains what they need in one clean document. It arrives as a recorded call, a WhatsApp thread, a PDF of their vendor terms, a screenshot of the spreadsheet they actually run on, and a paragraph in an email. Somebody has to read all of it and turn it into a plan — and that job goes wrong in a specific way: details get lost, and the gaps get filled with guesses. Months later the client says *we never asked for that*, and nobody can prove otherwise.

What goes in

Seven input types, and the format is detected from the file contents rather than its name, because clients rename things. Call transcripts, WhatsApp exports, PDFs, Word documents, screenshots, a web page by URL, and text typed or pasted straight in — which is the most common thing a client actually sends.

A screenshot is not a document you can search, so it is read by a vision model **at the moment it is uploaded** rather than at analysis time. That choice matters: turning the image into text once means every later stage treats it exactly like a transcript, and a claim drawn from a spreadsheet screenshot can be quoted and checked like any other. Passing raw images into each analysis call would put that content outside the verification path entirely.

The six AI calls

Pressing *Run discovery* makes six separate requests, not one.

- 1 · brief what they actually need, with every line quoted
- 2 · conflicts where their own documents contradict each other
- 3 · questions what nobody has answered yet
- 4 · process how it runs today, and what to change
- 5 · outline roles, modules, features, prioritised
- 6 · prototype a working demo of the tool being proposed

Six rather than one, for reasons that showed up in practice. Each is a different job, and a single request asking for all of it does every part worse. They fail independently — a broken prototype does not cost you the brief. Each is versioned separately, so you can re-run one thing. And the reader watches the work arrive rather than staring at a spinner.

Every call is told the exact shape of answer it must return, so what comes back is structured data rather than prose to be parsed hopefully.

The idea the whole thing rests on

AI makes things up. That is the known problem, and every serious objection to using it in consulting work comes back to it. So in this tool the AI writes the plan, but **it does not get to be believed**.

Every statement it produces must carry a verbatim quote from a source document. After it answers, the application goes back to the original text and checks that the quote is really

there — first an exact match, then a whitespace- and punctuation-normalised match, then a fuzzy match that is recorded but *not* counted as verified. Only the first two count.

The result is a grounding score, shown at the top of the brief rather than buried: **84% on the seeded demo — 27 of 32 statements verified**. Clicking any citation opens the source with the sentence highlighted in place.

It is deliberately not 100%, and that is the point

Four of the unverified statements are honest assumptions, marked as assumptions. The fifth is a claim I planted on purpose: it cites a sentence that appears in no document anywhere, and it renders red every single time.

That is the proof the check can actually fail. A verification layer that always says yes is decoration. There are tests asserting this from both ends — that the genuine citations pass, and that the planted one fails.

84%

grounded — 27 of 32, seeded demo

89%

a live run — 40 of 45

197

quotable spans across 6 documents

230

automated tests, plus 12 in a browser

Two things it finds that people miss

Contradictions. The founder says the first-phase budget is two lakh. Two weeks later the head of operations says five. Both are in the notes, nobody noticed, and a plan built on the wrong number fails. It shows both sides with a quote each and does not pick a winner, because that is the client's decision. Once recorded, the decision is fed back into the next run so it is not raised again.

Unanswered questions. Budget, timeline, integrations, what happens to eight months of spreadsheet history on the day the new system goes live. Each phrased so it can be pasted into an email. An absent answer is a finding — most tools quietly guess and move on.

The demo it builds

The sixth call does not describe what to build. It builds a working demo of it: a single self-contained page, seeded with the client's real project names and people, where actions have consequences that cross screens. A supervisor posts an update and it appears on the client's view; post something mentioning a cost and it stops in the founder's approval queue instead.

It runs inside a sandboxed frame that can execute its own code but cannot read anything from the application around it. The client does not just read the proposal — they can test it, and tell you it is wrong before anyone writes production code.

How the frontend and backend work together

One application rather than two. Pages are rendered on the server, which reads the database directly and sends back finished HTML — so the browser is never handed data it is not allowed to see, because the filtering happens before anything leaves the server. Changes (uploading, recording a decision, inviting a colleague) go through small API endpoints.

Generating a plan takes minutes, so that one path streams progress back as each stage lands, with a heartbeat every fifteen seconds to stop intermediaries treating a quiet connection as a dead one.

The business case, and scaling it

What it is actually worth

Discovery is the part of a project that is hardest to charge for and most expensive to get wrong. The time goes on reading and reconciling, and the risk sits in what gets missed. This does not replace the consultant — it removes the reading and, more importantly, it produces evidence.

WHERE IT PAYS HOW

Speed to proposal	Days of reading become minutes. A full analysis of six documents takes about eleven minutes end to end.
Fewer disputes	Every requirement is quoted back to the sentence it came from. "We never asked for that" becomes a link, not an argument.
Risk found early	Contradictions and gaps surface during discovery, when they are cheap, rather than during build.
Higher win rate	The client can operate the proposed tool before signing. A prototype they have used is a far stronger sell than one they have watched.
Scope control	Assumptions are listed separately and visibly, so the client corrects them at proposal stage instead of at delivery.

Who uses it

Three roles, and they are enforced rather than cosmetic. The **consultant** owns the workspace and runs the analysis. Their **colleagues** are invited into the same workspace and add to the same pile of evidence. The **client** gets a link — no account — showing the brief, the proposed process and the working demo, with a downloadable proposal. The client never sees the internal working notes: quoting their own staff disagreeing with each other is for the consultant, not for them.

Scaling it

The architecture was chosen so that growth is a configuration change rather than a rewrite.

More customers

Multi-tenant from the first table. Every row carries an organisation id, and separation is enforced **twice**: once in the application code, and again by row-level security policies inside the database, under a role that has no permission to bypass them. If the code asked wrongly, the database would still refuse. There are tests that try to break it and fail to. Adding a customer is a row, not an environment.

More load

The same schema runs on managed Postgres by setting one environment variable — the embedded database is a development convenience, not a different system, so row-level security, indexes and transactions behave identically. Deployment is configured for Vercel, with migrations running as part of the build.

More documents per engagement

This is where I would be honest about the ceiling. Today the whole corpus is sent in one context window, which is the right call for the handful of incompatible documents a real client sends, and it is why cross-document synthesis works at all — a retrieval system would chunk the evidence and make exactly this job worse. At a few hundred documents that reverses, and the answer is a retrieval layer feeding the same six calls. The verification layer would not change, which is the part that matters.

Cost

The model is configurable per deployment. Sonnet is the default because the work is careful reading rather than deep reasoning, and cost per run decides whether a consultant re-runs after every new document — which is the behaviour the product wants. The five independent stages run concurrently, so wall-clock time is roughly the longest stage rather than the sum of all six.

An honest note on efficiency

Prompt caching is set up correctly and currently never pays off. I measured it rather than assuming: every stage reports zero cache reads, because the structured-output schema sits ahead of the system prompt in the cached prefix, and the six stages each need their own schema.

Making it work would mean one shared schema validated only after the fact. I kept the per-stage schemas, because a guaranteed-shape response is what makes every downstream stage parseable in a product whose entire claim is that its output can be checked. It is documented in the README, because an efficiency claim nobody verified is worse than no claim at all.

What I would build next

- **Retrieval**, once an engagement outgrows a single context window.
- **Background jobs**. Analysis currently runs inside the request. A queue would survive a closed laptop and allow retrying a single failed stage.
- **Real invitation emails**. Today it produces a link you send yourself.
- **Client replies on the questions**, so the answers arrive in the tool rather than in an inbox.

What can go wrong when you run it

This was built inside a limited time as a proposed idea rather than a production system, so here are the rough edges I know about, what causes them, and what to do. Everything below has an error message in the application that says the same thing — none of them fail silently.

"No ANTHROPIC_API_KEY is set"

Cause: no key, or a key added while the server was running. `.env` is read once at startup.

Fix: `npm run set-key`, then restart. Or ignore it — the seeded demo works fully without a key.

The recorded analysis is deliberately refused for *your own* documents, because replaying an analysis of one company's project over another company's files would be dishonest.

"The Anthropic account has no capacity left"

Cause: the API key has hit its usage limit. *Fix:* nothing in the code can work around it. The seeded demo still runs.

"The outline answer was cut off after N tokens"

Cause: a stage produced more output than its budget allowed — a large corpus makes a large brief. *Fix:* re-run; everything that completed has been saved. Budgets were sized from measurement, but a much bigger corpus could exceed them.

"The local database is already open in process N"

Cause: the embedded database allows one process at a time, and something else has it open — usually a second `npm run dev`. *Fix:* stop the other one, or set `PGLITE_DATA_DIR` to a different folder. This message exists because opening it twice corrupts the data silently, and I would rather refuse loudly.

A page shows an empty workspace, and saving fails

Cause: a browser session left over from before `npm run db:reset` — the ids in the cookie no longer exist. *Fix:* sign out and in. Sessions are now re-checked against the database on every request, so this should send you to the sign-in screen rather than showing an empty page.

An uploaded screenshot shows "0 spans"

Cause: reading an image needs a model, so with no API key it is stored but not transcribed.

Fix: set a key and upload it again — transcription happens at upload, so the stored copy stays empty. The source list says exactly this next to the file.

Generation takes about eleven minutes

Not a fault, but worth expecting. It is six calls over the whole corpus and the final one writes a complete working web page. Progress streams stage by stage so you can watch it.

Deliberately not built

- **Not deployed.** Configured for Vercel; it has only ever run locally.
- **No background job queue.** Analysis runs inside the request, so closing the laptop mid-run loses the stages still in flight — the ones already finished are saved.
- **No email delivery.** Invitations produce a link to send yourself.
- **No file storage service.** Uploads are parsed to text and stored in the database; the original binary is not kept, apart from screenshots.
- **No rate limiting or audit log.** Both matter in production and prove nothing about the parts being assessed here.

Happy to walk through any of this, or to go deeper on the verification layer and the tenant isolation — those are the two parts I would most want to be questioned on.